

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №3
По дисциплине "Вычислительная математика"
Вариант №2

Выполнил студент группы Р3209
Евграфов Артём Андреевич
Проверил:
Рыбаков Степан Дмитриевич

Санкт-Петербург 2026

Содержание

1. Вычислительная реализация	2
2. Листинг программы	3
3. Вывод	6

1. Вычислительная реализация

1. Точное вычисление

$$\int_{-3}^{-1} (-3x^3 - 5x^2 + 4x - 2)dx = \left(-\frac{3x^4}{4} - \frac{5x^3}{3} + \frac{4x^2}{2} - 2x \right) \Big|_{-3}^{-1} = \frac{59}{12} - \frac{99}{12} = -\frac{10}{3} \approx -3,333$$

2. По формуле Ньютона-Котеса при n=6

(Примечание: шаг $h = \frac{-1-(-3)}{6} = \frac{1}{3}$, $b - a = 2$, коэффициенты c_6^i умножаются на $b - a$)

$$\begin{aligned} \int_{-3}^{-1} (-3x^3 - 5x^2 + 4x - 2)dx &= \sum_{i=0}^n f(x_i)c_n^i = \\ &= f(-3)c_6^0 + f\left(-3 + \frac{1}{3}\right)c_6^1 + f\left(-3 + \frac{2}{3}\right)c_6^2 + f\left(-3 + \frac{3}{3}\right)c_6^3 + \\ &\quad + f\left(-3 + \frac{4}{3}\right)c_6^4 + f\left(-3 + \frac{5}{3}\right)c_6^5 + f(-1)c_6^6 = \\ &= \frac{41 \cdot 2}{840}(f(-3) + f(-1)) + \frac{216 \cdot 2}{840}\left(f\left(-\frac{8}{3}\right) + f\left(-\frac{4}{3}\right)\right) + \\ &\quad + \frac{27 \cdot 2}{840}\left(f\left(-\frac{7}{3}\right) + f\left(-\frac{5}{3}\right)\right) + \frac{272 \cdot 2}{840}f(-2) = \\ &= \frac{41}{420}(22 - 8) + \frac{216}{420}\left(\frac{26}{3} - \frac{82}{9}\right) + \frac{27}{420}\left(-\frac{4}{9} - \frac{26}{3}\right) + \frac{272}{420}(-6) = \\ &= -\frac{10}{3} \approx -3,333 \end{aligned}$$

Относительная погрешность: $\frac{|-3,333 - (-3,333)|}{|-3,333|} * 100\% = 0\%$

3. Средние прямоугольники

$$n = 10; h = \frac{b - a}{n} = \frac{-1 - (-3)}{10} = 0,2$$

i	1	2	3	4	5	6	7	8	9	10
$x_{i-\frac{1}{2}}$	-2,9	-2,7	-2,5	-2,3	-2,1	-1,9	-1,7	-1,5	-1,3	-1,1
$y_{i-\frac{1}{2}}$	17,517	9,799	3,625	-1,149	-4,667	-7,073	-8,511	-9,125	-9,059	-8,457

Сумма: $-17,100 * 0,2 = -3,420$

Относительная погрешность: $\frac{|-3,333 - (-3,420)|}{|-3,333|} * 100\% = 2,61\%$

4. Трапеции

i	0	1	2	3	4	5	6	7	8	9	10
x_i	-3,0	-2,8	-2,6	-2,4	-2,2	-2,0	-1,8	-1,6	-1,4	-1,2	-1,0
y_i	22	13,456	6,528	1,072	-3,056	-6	-7,904	-8,912	-9,168	-8,816	-8

Сумма: $0,1(22 - 8 + 2 \cdot 13,456 + 2 \cdot 6,528 + 2 \cdot 1,072 - 2 \cdot 3,056 - 2 \cdot 6 - 2 \cdot 7,904 - 2 \cdot 8,912 - 2 \cdot 9,168 - 2 \cdot 8,816) = -3,160$

Относительная погрешность: $\frac{|-3,333 - (-3,160)|}{|-3,333|} * 100\% = 5,19\%$

5. По формуле Симпсона

$$\begin{aligned} \int_{-3}^{-1} (-3x^3 - 5x^2 + 4x - 2)dx &= \\ &= \frac{0,2}{3}[(22 - 8) + 4(13,456 + 1,072 - 6 - 8,912 - 8,816) + \\ &\quad + 2(6,528 - 3,056 - 7,904 - 9,168)] = \\ &= \frac{0,2}{3}[14 + 4(-9,200) + 2(-13,600)] = -\frac{10}{3} \approx -3,333 \end{aligned}$$

Относительная погрешность: $\frac{|-3,333 - (-3,333)|}{|-3,333|} * 100\% = 0\%$

2. Листинг программы

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  struct MathFunction {
6      string name;
7      function<double(double)> calc;
8      bool has_singularity;
9      double singularity_point;
10     bool converges;
11 };
12
13 double f1(double x) { return -3 * pow(x, 3) - 5 * pow(x, 2) + 4 * x - 2; }
14 double f2(double x) { return 1.0 / sqrt(x); }
15 double f3(double x) { return 1.0 / (x - 1.0); }
16
17 vector<MathFunction> functions_db = {
18     {"-3x^3 - 5x^2 + 4x - 2", f1, false, 0.0, true},
19     {"1 / sqrt(x)", f2, true, 0.0, true},
20     {"1 / (x - 1)", f3, true, 1.0, false}
21 };
22
23 double left_rectangles(function<double(double)> f, double a, double b, int n) {
24     double h = (b - a) / n;
25     double sum = 0;
26     for (int i = 0; i < n; i++) {
27         sum += f(a + i * h);
28     }
29     return h * sum;
30 }
31
32 double right_rectangles(function<double(double)> f, double a, double b, int n) {
33     double h = (b - a) / n;
34     double sum = 0;
35     for (int i = 1; i <= n; i++) {
36         sum += f(a + i * h);
37     }
38     return h * sum;
39 }
40
41 double middle_rectangles(function<double(double)> f, double a, double b, int n) {
42     double h = (b - a) / n;
43     double sum = 0;
44     for (int i = 0; i < n; i++) {
45         sum += f(a + i * h + h / 2.0);
46     }
47     return h * sum;
48 }
49
50 double trapezoidal(function<double(double)> f, double a, double b, int n) {
51     double h = (b - a) / n;
52     double sum = (f(a) + f(b)) / 2.0;
53     for (int i = 1; i < n; i++) {
54         sum += f(a + i * h);
55     }
56     return h * sum;
57 }
58
```

```

59 double simpson(function<double(double)> f, double a, double b, int n) {
60     double h = (b - a) / n;
61     double sum = f(a) + f(b);
62     double sum_odd = 0;
63     double sum_even = 0;
64     for (int i = 1; i < n; i++) {
65         if (i % 2 == 0) sum_even += f(a + i * h);
66         else sum_odd += f(a + i * h);
67     }
68     return (h / 3.0) * (sum + 2 * sum_even + 4 * sum_odd);
69 }
70
71 struct RungeResult {
72     double value;
73     int n_final;
74 };
75
76 RungeResult integrate_with_runge(
77     function<double(function<double(double)>, double, double, int)> method,
78     function<double(double)> f, double a, double b, double eps, int k)
79 {
80     int n = 4;
81     double I_prev = method(f, a, b, n);
82     double I_curr;
83
84     while (true) {
85         n *= 2;
86         I_curr = method(f, a, b, n);
87         double error = abs(I_curr - I_prev) / (pow(2, k) - 1);
88         if (error <= eps) {
89             break;
90         }
91         I_prev = I_curr;
92         if (n > 1000000) {
93             break;
94         }
95     }
96     return {I_curr, n};
97 }
98 template <typename T>
99 T get_input(const string& prompt) {
100     T value;
101     while (true) {
102         cout << prompt;
103         while (!(cin >> value)) {
104             cin.clear();
105             cin.ignore(numeric_limits<streamsize>::max(), '\n');
106             cout << "Некорректный ввод. Повторите: ";
107         }
108         cin.clear();
109         cin.ignore(numeric_limits<streamsize>::max(), '\n');
110         return value;
111     }
112 }
113
114 int main() {
115     int func_choice;
116     while (true) {
117         cout << "Выберите функцию:\n";
118         for (size_t i = 0; i < functions_db.size(); i++) {
119             cout << i + 1 << ". " << functions_db[i].name << "\n";

```

```

120     }
121     func_choice = get_input<int>("Ввод: ");
122     if (func_choice >= 1 && func_choice <= 3) break;
123     cout << "Неверный выбор функции.\n";
124 }
125
126 MathFunction selected_func = functions_db[func_choice - 1];
127
128 double a = get_input<double>("Введите a: ");
129 double b = get_input<double>("Введите b: ");
130 bool flag = false;
131 if (a > b) {
132     flag = true;
133     swap(a, b);
134 }
135
136 double eps;
137 while (true) {
138     eps = get_input<double>("Введите точность eps: ");
139     if (eps > 0) break;
140     cout << "Точность должна быть больше 0\n";
141 }
142
143 vector<pair<double, double>> intervals;
144
145 if (selected_func.has_singularity && selected_func.singularity_point >= a &&
146     → selected_func.singularity_point <= b) {
147     if (!selected_func.converges || (func_choice == 2 && min(a, b) < 0)) {
148         cout << "\nИнтеграл не существует\n";
149         return 0;
150     }
151
152     double delta = 1e-5;
153     double sp = selected_func.singularity_point;
154
155     if (abs(a - sp) < 1e-9) {
156         intervals.push_back({a + delta, b});
157     }
158     else if (abs(b - sp) < 1e-9) {
159         intervals.push_back({a, b - delta});
160     }
161     else {
162         intervals.push_back({a, sp - delta});
163         intervals.push_back({sp + delta, b});
164     }
165 } else {
166     intervals.push_back({a, b});
167 }
168
169 int method_choice;
170 while (true) {
171     cout << "\nВыберите метод:\n";
172     cout << "1. Левые прямоугольники\n";
173     cout << "2. Правые прямоугольники\n";
174     cout << "3. Средние прямоугольники\n";
175     cout << "4. Трапеции\n";
176     cout << "5. Симпсон\n";
177     method_choice = get_input<int>("Ввод: ");
178     if (method_choice >= 1 && method_choice <= 5) break;
179     cout << "Неверный выбор метода.\n";
180 }

```

```

180
181 function<double(function<double(double)>), double, double, int> numerical_method;
182 int k_order = 2;
183
184 switch (method_choice) {
185     case 1: numerical_method = left_rectangles; break;
186     case 2: numerical_method = right_rectangles; break;
187     case 3: numerical_method = middle_rectangles; break;
188     case 4: numerical_method = trapezoidal; break;
189     case 5: numerical_method = simpson; k_order = 4; break;
190 }
191
192 double final_integral_value = 0;
193 int max_n_used = 0;
194
195 for (auto& interval : intervals) {
196     RungeResult res = integrate_with_runge(
197         numerical_method, selected_func.calc,
198         interval.first, interval.second, eps, k_order
199     );
200     final_integral_value += res.value;
201     max_n_used = max(max_n_used, res.n_final);
202 }
203 if (flag) {
204     final_integral_value *= -1;
205 }
206 cout << fixed << setprecision(6);
207 cout << "\nЗначение интеграла: " << final_integral_value << "\n";
208 cout << "Число разбиений (n): " << max_n_used << "\n\n";
209
210 return 0;
211 }
212
213

```

3. Вывод

В ходе выполнения лабораторной работы я изучил и программно реализовал основные методы численного интегрирования: методы прямоугольников (левых, правых, средних), метод трапеций, метод Симпсона, а также применил формулу Ньютона-Котеса. Как показали аналитические вычисления, самыми точными оказались метод Симпсона и метод Ньютона-Котеса: поскольку в качестве тестовой функции выступал многочлен третьей степени, эти методы выдали абсолютно точный результат (погрешность 0) даже при небольшом числе интервалов. Кроме того, в процессе программирования я освоил применение правила Рунге.